CS 380: ARTIFICIAL INTELLIGENCE

Lecture 5: Local Search & CSPs

Recall: Problem Solving

- Idea: represent the problem we want to solve as:
  - State space
  - Actions
  - Goal Check
  - Heuristic / evaluation function
- If we do this, we can then use search methods to solve any problem in that space
  - BFS, DFS, Iterative Deepening, Greedy Search, A*

Systematic vs. Local Search

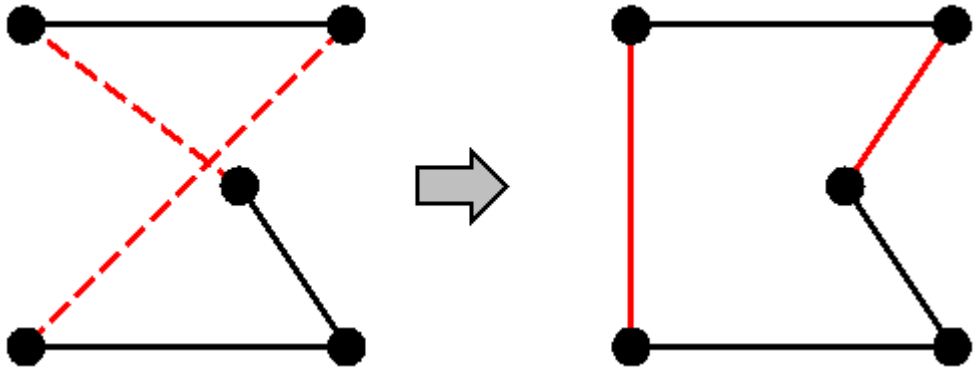
- All the algorithms we have seen so far have been "systematic":
  - They will systematically explore ALL the states in the search space if needed
    - Each of them just explores the states in a different order
  - If the search space is finite, they are all complete
- Local Search:
  - Instead of systematically exploring the state space…
  - Start with one state (even a bad one) and iteratively improve it by exploring neighbor states ("local")
  - Search space is the set of all states visited
  - *Iterative improvement*

Iterative Improvement Algorithms

- In many optimization problems, path is irrelevant
  - Goal state itself is the solution
- Then state space = set of "complete" configurations
  - Find optimal configuration, e.g., TSP
  - Or, find configuration satisfying constraints, e.g., timetable
- In such cases, can use iterative improvement algorithms
  - keep a single "current" state and try to improve it
- Constant space, suitable for online as well as offline search

Example: Traveling Salesperson Problem

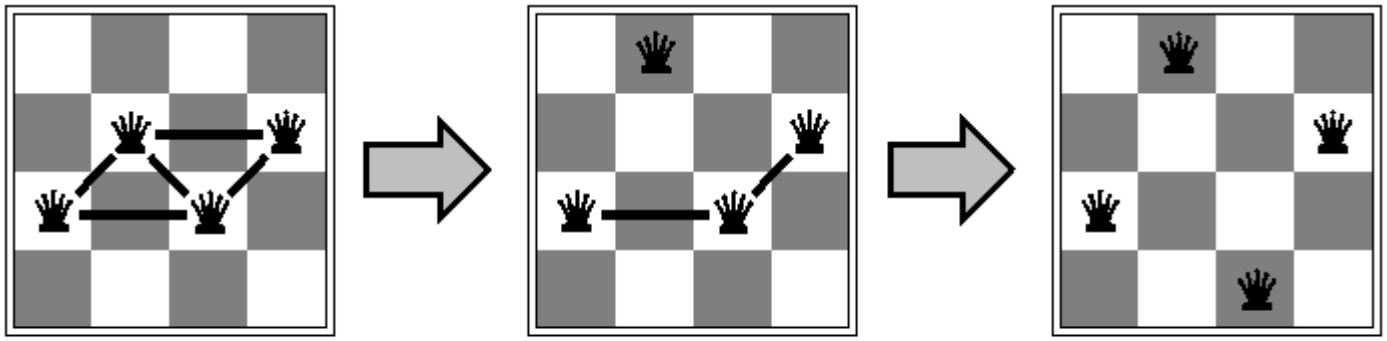
- Start with any complete tour, perform pairwise exchanges



- Variants of this approach get within 1% of optimal very quickly with thousands of cities

Example: n-queens

- Put *n* queens on an *n* x *n* board with no two queens on the same row, column, or diagonal
- Move a queen to reduce the number of conflicts



- Almost always solves n-queens problems almost instantaneously for very large *n*, e.g., *n* = 1 million

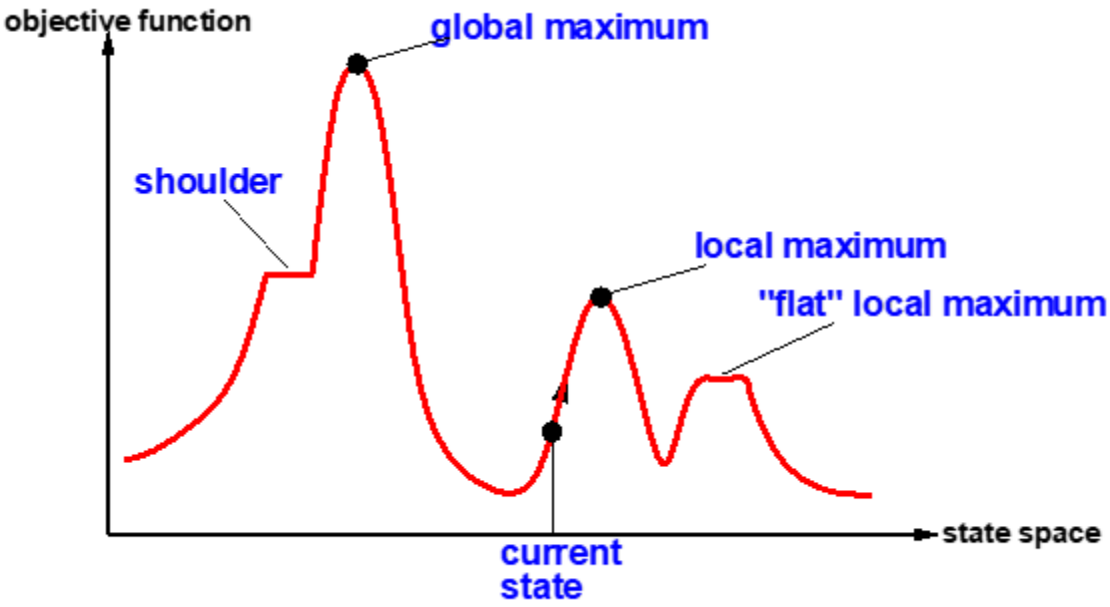
Hill-climbing (or gradient ascent/descent)

- "Like climbing Everest in think fog with amnesia"

```
function Hill-Climbing(problem) returns a state that is a local maximum
    inputs: problem, a problem
    local variables: current, a node
                     neighbor, a node
    current ← Make-Node(Initial-State[problem])
    loop do
        neighbor ← a highest-valued successor of current
        if Value[neighbor] ≤ Value[current] then return State[current]
        current ← neighbor
    end
```

Hill-climbing cont.

- Useful to consider state space landscape



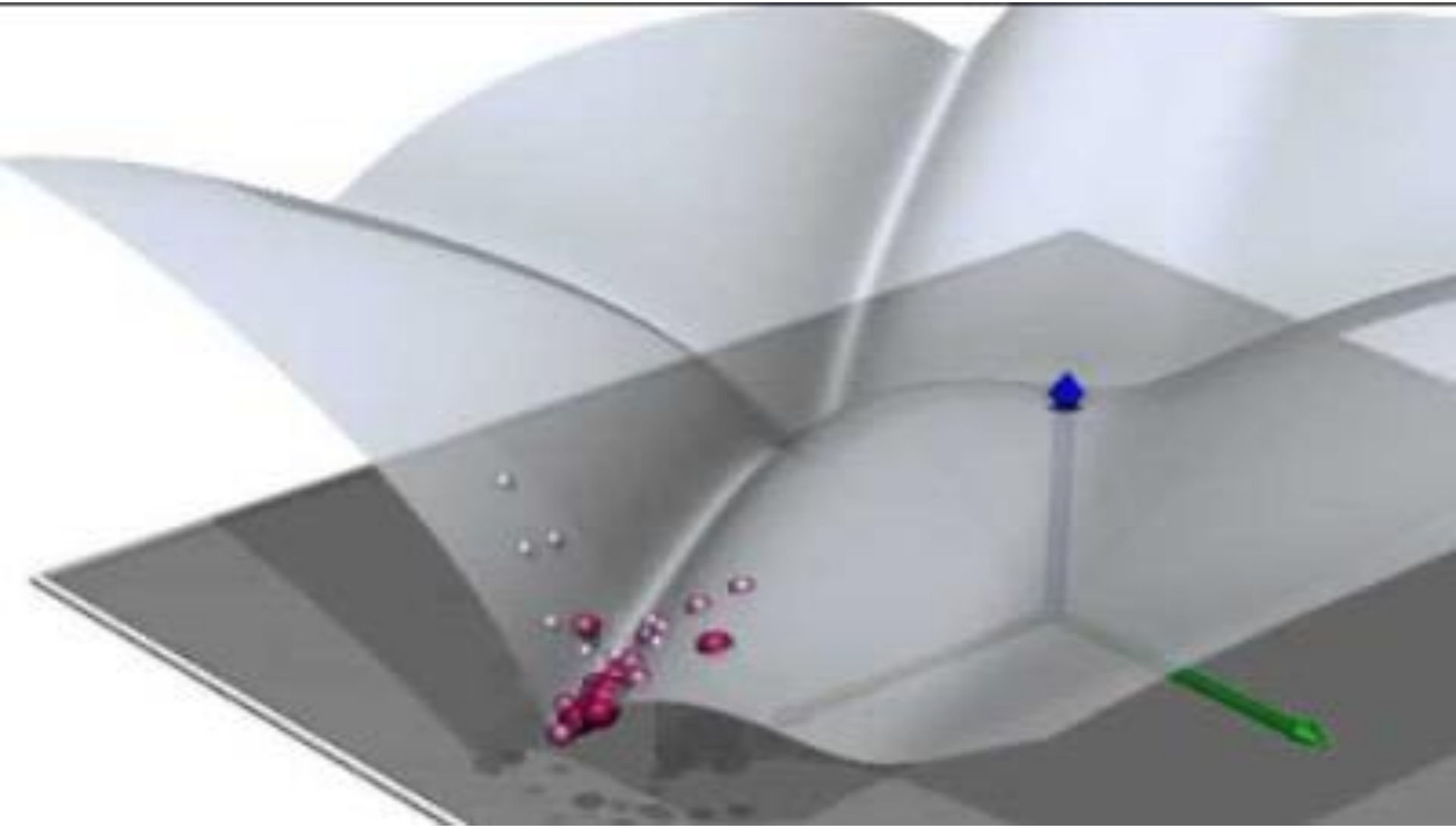
- Random-restart hill climbing overcomes local maxima – trivially complete
- Random sideways moves escape from shoulders but loop on flat maxima

Simulated Annealing

- "Annealing" = heating and then controlled cooling (of metals to strengthen the material)
- At a fixed temperature *T*, the state occupation probability reaches the Boltzmann distribution

$$p(x) = \alpha e^{\frac{E(x)}{kT}}$$

- If *T* is decreased slowly enough, you are guaranteed to reach the optimal state (if we need optimal)
- Visualization:
  - https://www.youtube.com/watch?v=iaq_Fpr4KZc



Simulated Annealing

- Idea: escape local maxima by allowing some "bad" moves
- But gradually decrease their size and frequency

```
function Simulated-Annealing(problem, schedule) returns a solution state
   inputs: problem, a problem
           schedule, a mapping from time to "temperature"
   local variables: current, a node
                    next, a node
                    T, a "temperature" controlling prob. of downward steps

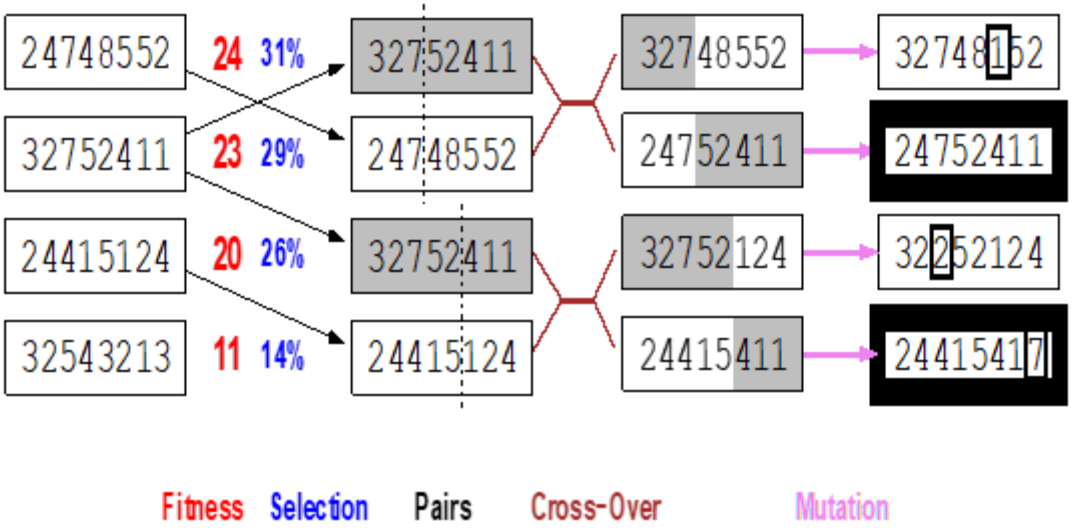
   current ← Make-Node(Initial-State[problem])
   for t ← 1 to ∞ do
       T ← schedule[t]
       if T = 0 then return current
       next ← a randomly selected successor of current
       ΔE ← Value[next] – Value[current]
       if ΔE > 0 then current ← next
       else current ← next only with probability e^(Δ E/T)
```

Local Beam Search

- Idea: Keep *k* states instead of 1; choose top *k* of all their successors
- Not the same as *k* searches run in parallel!
- Searches that find good states recruit other searches to join them
- Problem: quite often, all *k* states end up on same local hill
- Idea: choose *k* successors randomly, biased towards the good ones
- Observe the analogy to natural selection.

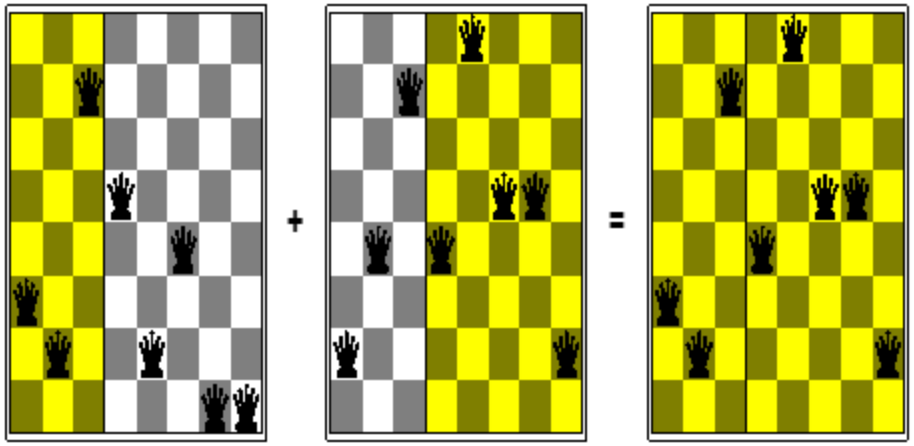
Genetic Algorithms

- Stochastic local beam search + generate successor from pairs of states



Genetic Algorithms Cont.

- GAs require states encode a strings (GPs use programs)
- Crossover helps iff substrings are meaningful components



- GAs != evolution: e.g., real genes encode replication machinery.

Sample Visualization

- Greedy search, local search, simulated annealing…

https://www.youtube.com/watch?v=SC5CX8drAtU



Constraint Satisfaction Problems (CSPS)

- Standard search problem:
  - State is a "black box" – any old data structure that supports goal test, eval, successor
- CSP:
  - State is defined by *variables* V_i with *values* from *domain* D_i
  - Goal test is a set of *constraints* specifying allowable combinations of values for subsets of variables
  - Simple example of a *formal representation language*
  - Allows useful *general-purpose* algorithms with more power than standard search algorithms

Example: 4-Queens as a CSP

- Assume 1 queen in each column. Which row does each one go in?

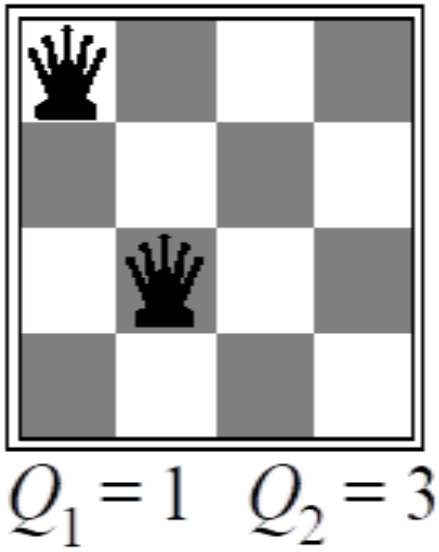
Variables: Q_1, Q_2, Q_3, Q_4

Domains: $D_i = \{1,2,3,4\}$

Constraints:

Q_i != Q_j (cannot be in same row)

$|Q_i - Q_j|$!= $|i - j|$ (or same diagonal)

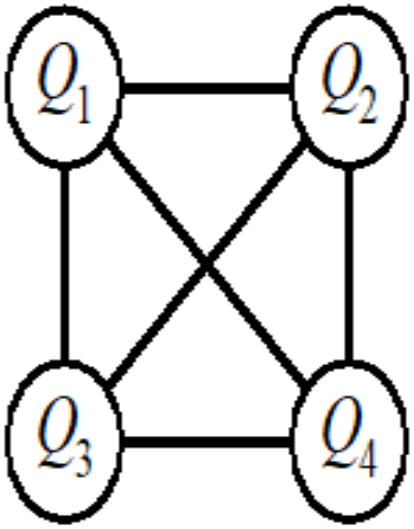


Translate each constraint into set of allowable values for its variables

E.g., values for (Q_1, Q_2) are (1,3) (1,4) (2,4) (3,1) (4,1) (4,2)

Constraint Graph

- Binary CSP: each constrain relates to at most two variables
- Constraint graph: nodes are variables, arcs show constraints



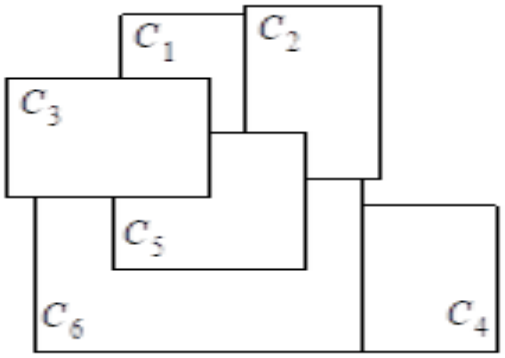
Example: Cryptarithmetic

- Variables
  - *D E M N O R S Y*
- Domains
  - {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
- Constraints
  - *M* != 0, *S* != 0 (unary constraints)
  - *Y* = *D* + *E* or *Y* = *D* + *E* – 10, etc.
  - *D* != *E*, *D* != *M*, *D* != *N*, etc.

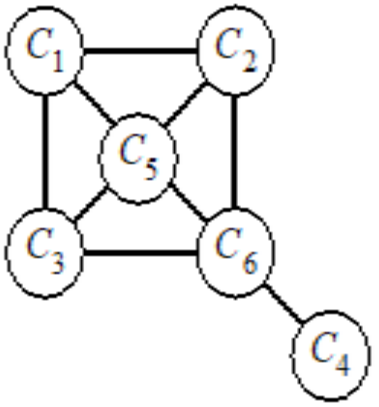


Example: Map Coloring

- Color a map so that no adjacent countries have the same color
- Variables
  - Countries C_i
- Domains
  - {Red, Blue, Green}
- Constraints
  - C_1 != C_2, C_2 != C_5, etc.



Constraint Graph:



Real-world CSPs

- Assignment problems
  - e.g., who teaches what class
- Timetabling problems
  - e.g., which class is offered when and where?
- Hardware configuration
- Spreadsheets
- Transportation scheduling
- Factory scheduling
- Floorplanning

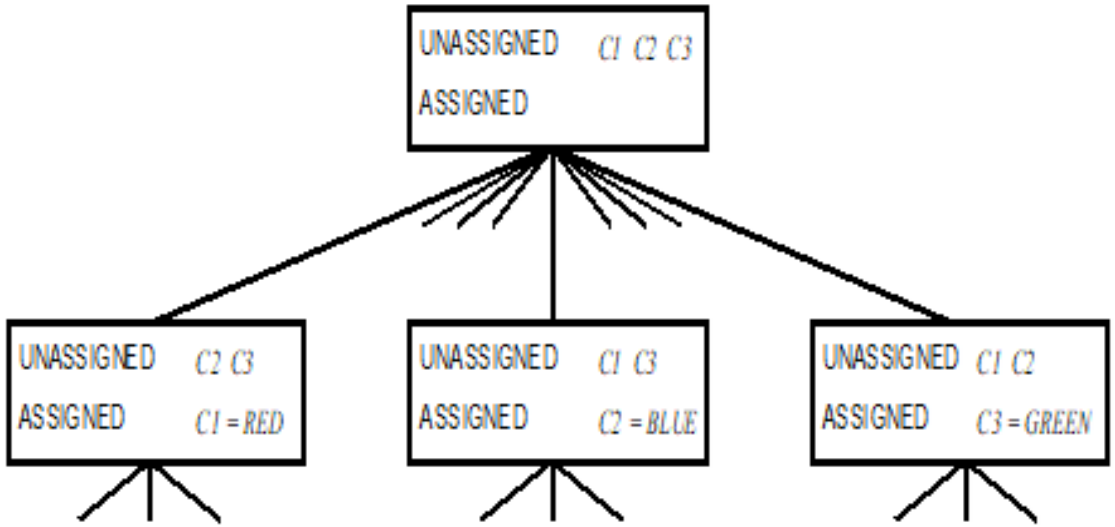
Notice that many real-world problems involve real-valued variables

Applying standard search

- Start with straightforward, “dumb” approach, then fix it
- States are defined by the values assigned so far
- Initial State: all variables unassigned
- Actions: assign a value to an unassigned variable
- Goal test: all variables assigned, no constraints violated

- Notice that this is the same for all CSPs

Standard search applied to map-coloring



Complexity of the dumb approach

- Max. depth of space m: ??
- Depth of solution state d: ??
- Search algorithm to use: ??
- Branching factor b: ??

Complexity of the dumb approach

- Max. depth of space m: *n* (number of variables)
- Depth of solution state d: *n* (all vars assigned)
- Search algorithm to use: depth-first
- Branching factor b: $\sum_i |Di|$ (at top of tree)

  $|\ D_i\ |$ = size of Domain *i*
- This can be improved dramatically by noting the following:
  - Order of assignment is irrelevant, hence many paths are equivalent
  - Adding assignments cannot correct a violated constraint

Backtracking Search

Use depth-first search, but

1) Fix the order of assignment => b = $|D_i|$

   (can be done in the SUCCESSORS function)

2) Check for constraint violations

The constraint violation check can be implemented in two ways:

1) Modify SUCCESSORS to assign only values that are allowed, given the values already assigned
2) Check constraints are satisfied before expanding a state

Backtracking search is the basic uninformed algorithm for CSPs

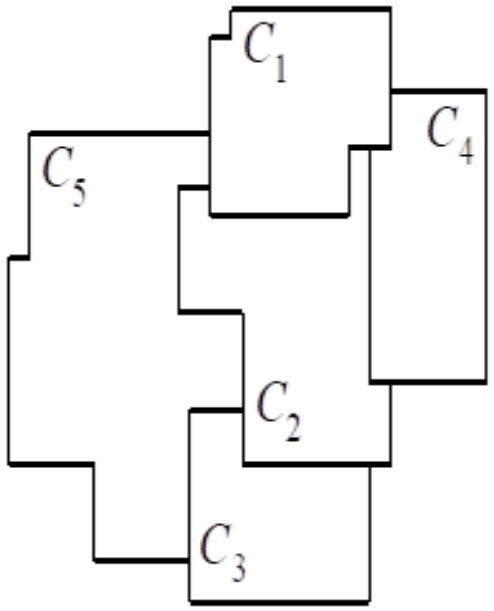
Can solve *n*-queens for $n \approx 15$

Forward Checking

- Idea: Keep track of remaining legal values for unassigned variables.
- Terminate search when any variable has no legal values.

Simplified map-coloring example:

	Red	Blue	Green
C_1			
C_2			
C_3			
C_4			
C_5			



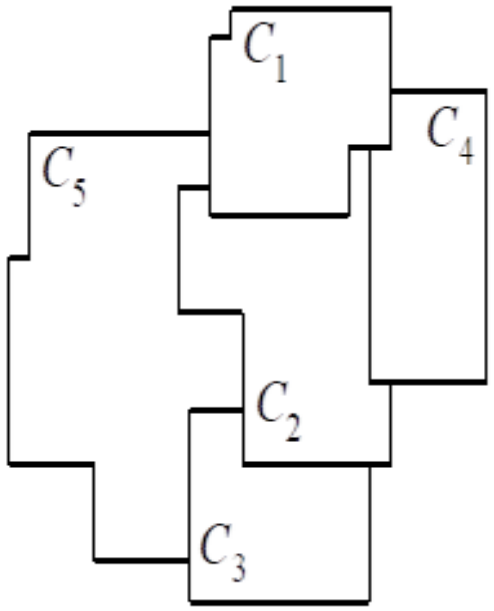
Can solve n-queens up to n ≈ 30

Forward Checking

- Idea: Keep track of remaining legal values for unassigned variables.
- Terminate search when any variable has no legal values.

Simplified map-coloring example:

	Red	Blue	Green
C_1	✓		
C_2	x		
C_3			
C_4	x		
C_5	x		



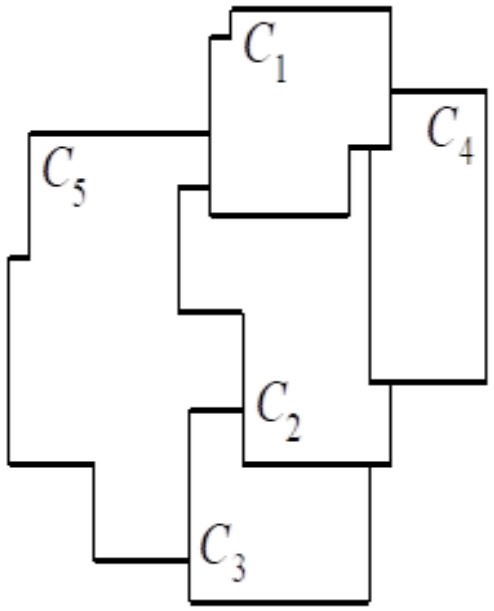
Can solve n-queens up to n ≈ 30

Forward Checking

- Idea: Keep track of remaining legal values for unassigned variables.
- Terminate search when any variable has no legal values.

Simplified map-coloring example:

	Red	Blue	Green
C_1	✓		
C_2	x	✓	
C_3		x	
C_4	x	x	
C_5	x	x	



Can solve n-queens up to n ≈ 30

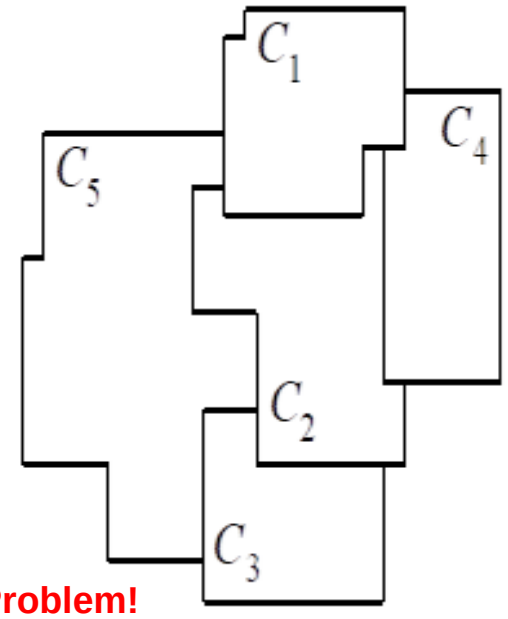
Forward Checking

- Idea: Keep track of remaining legal values for unassigned variables.
- Terminate search when any variable has no legal values.

Simplified map-coloring example:

	Red	Blue	Green
C_1	✓		
C_2	X	✓	
C_3		X	✓
C_4	X	X	
C_5	X	X	X

← Problem!



Can solve n-queens up to n ≈ 30

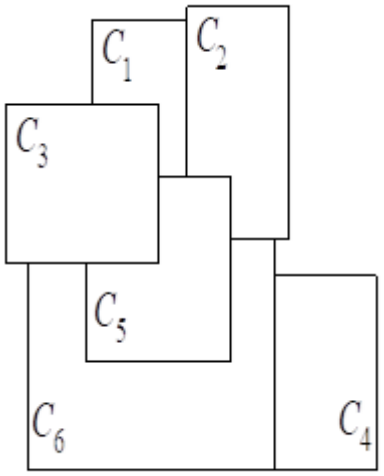
Heuristics for CSPs

- More intelligent decision on which value to choose for each variable, which variable to assign next

Given C_1 = Red, C_2 = Green, chose C_3 = ??

Given C_1 = Red, C_2 = Green, what next??

Can solve n-queens for n ≈ 1000



Heuristics for CSPs

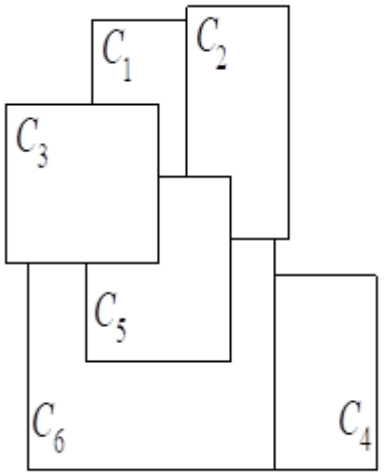
- More intelligent decision on which value to choose for each variable, which variable to assign next

Given C_1 = Red, C_2 = Green, chose C_3 = ??

C_3 = Green: Least constraining value

Given C_1 = Red, C_2 = Green, what next??

C_5 : Most constrained variable



Can solve n-queens for n ≈ 1000

Iterative algorithms for CSPs

- Hill-climbing, simulated annealing typically work with "complete" states, i.e., all variables assigned
- To apply to CSPs:
  - Allow states with unsatisfied constraints
  - Operators reassign variable values
- Variable selection: randomly select any conflicted variable
- Min-conflicts heuristic:
  - Choose value that violates the fewest constraints
  - i.e., hillclimb with *h(n)* = total number of violated constraints

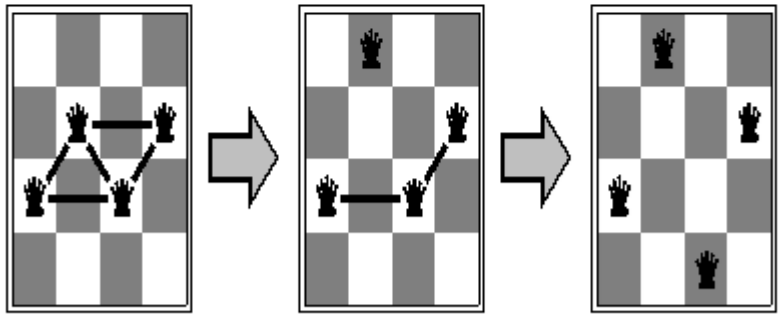
Example: 4-Queens

States: 4 queens in 4 columns ($4^4 = 256$ states)

Operators: move queen in column

Goal test: no attacks

Evaluation: $h(n)$ = number of attacks

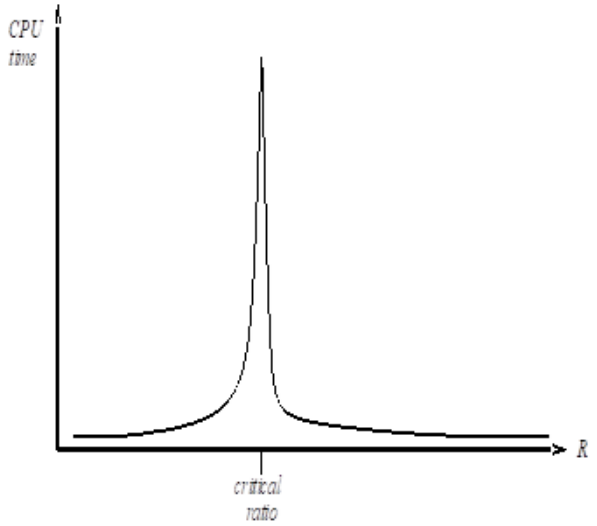


Performance of min-conflicts

Given random initial state, can solve n-queens in almost constant time for arbitrary n with high probability (e.g., $n = 10{,}000{,}000$)

The same appears to be true for any randomly-generated CSP except in a narrow range of the ratio

$$R = \frac{\text{number of constraints}}{\text{number of variables}}$$



Summary

- CSPs are a special kind of problem:
  - States defined by values of a fixed set of variables
  - Goal test defined by constraints on variable values
- Backtracking = depth-first search with
  - Fixed variable order
  - Only legal successors
- Forward checking prevents assignments that guarantee later failure
- Variable ordering and value selection heuristics help significantly
- Iterative min-conflicts is usually effective in practice
- Tree-structured CSPs can always be solved very efficiently

Coming up…

- We will start applying ideas of search to adversarial domains, where one player is playing another